

privateSQL_tpch_eval_customer_policy

January 7, 2026

1 TPC-H + PrivateSQL (Customer policy) — Notebook Overview

This notebook evaluates PrivateSQL-style differential privacy on TPC-H queries using a “customer” privacy policy. It loads TPC-H tables, builds DP queries, runs many DP trials, computes error metrics, and plots results.

```
[1]: import os
os.environ["JAVA_HOME"] = "/opt/homebrew/opt/openjdk@17/libexec/openjdk.jdk/
↳Contents/Home"
os.environ["SPARK_HOME"] = "/Users/kinchintong/project/tumult-labs/spark-4.0.
↳1-bin-hadoop3"

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import findspark
findspark.init(os.environ["SPARK_HOME"])
from math import floor
from pyspark import SparkFiles
from pyspark.sql import SparkSession, types as T, functions as F
from tmlt.analytics.privacy_budget import PureDPBudget
from tmlt.analytics.protected_change import (AddOneRow, AddMaxRows,
↳AddRowsWithID)
from tmlt.analytics.query_builder import QueryBuilder
from tmlt.analytics.session import Session, ColumnType
from tmlt.analytics.keyset import KeySet
from functools import reduce
from tmlt.analytics import (
    AddOneRow,
    PureDPBudget,
    QueryBuilder,
    Session,
    AddRowsWithID,
    GroupbyCountQuery,
)
```

```

from tmlt.analytics.truncation_strategy import TruncationStrategy

# Silence some Spark warnings
import warnings
warnings.simplefilter('ignore', UserWarning)
warnings.simplefilter('ignore', ResourceWarning)

findspark.init()

# Set up Spark
spark = (
    SparkSession.builder
        .appName("tpch-setup")
        .master("local[*]")          # drop if connecting to a cluster
        .config("spark.sql.adaptive.enabled", "true")
        .config("spark.driver.memory", "16g")
        .config("spark.executor.memory", "16g")
        .config("spark.driver.maxResultSize", "2g")
        .getOrCreate()
)
print("Spark version:", spark.version)

spark.sparkContext.setLogLevel("ERROR")

spark.conf.set("spark.sql.session.timeZone", "UTC")

# CREATE EMPTY TABLES (schema only) ----
def create_empty_tables():
    for t in TABLES:
        empty = spark.createDataFrame([], SCHEMAS[t])
        # managed Parquet tables; change to .format("delta") if using Delta
        # empty.write.mode("overwrite").format("parquet").saveAsTable(t)
        print(f"Created empty schema in database `{DB}`.")

# LOAD FROM CSV / .tbl (pipe-delimited with trailing '/') ----
def load_from_csv(src_dir: str):
    for t in TABLES:
        schema = SCHEMAS[t]
        path = f"{src_dir}/{t}.tbl"    # change to .csv if needed
        #only if the path exists
        if not os.path.exists(path):
            print(f"Path does not exist: {path}")
            continue

        print(f"Loading {t} from {path}...")

```

```

        # create a temporary wide schema of string columns (named) to absorb
↪trailing '|'
        n_cols = len(schema.fields) + 1
        tmp_fields = [T.StructField(f"_c{i}", T.StringType(), True) for i in
↪range(n_cols)]
        tmp_schema = T.StructType(tmp_fields)
        #print("tmp_schema:", tmp_schema)
        df_raw = (spark.read
                    .option("sep", "|")
                    .option("header", "false")
                    .option("mode", "PERMISSIVE")
                    .schema(tmp_schema) # read wide to absorb trailing '|'
                    .csv(path))

        #print(f"Raw schema for {t}:")
        #df_raw.printSchema()

        # keep only first N columns and cast to target schema
        cols = df_raw.columns[:len(schema.fields)]
        df = df_raw.select([F.col(c).alias(schema.fields[i].name) for i, c in
↪enumerate(cols)])

        # cast columns to target types
        for f in schema.fields:
            if isinstance(f.dataType, T.DateType):
                df = df.withColumn(f.name, F.to_date(F.col(f.name),
↪"yyyy-MM-dd"))
            elif isinstance(f.dataType, T.DecimalType):
                df = df.withColumn(f.name, F.col(f.name).cast(f.dataType))
            elif isinstance(f.dataType, T.IntegerType):
                df = df.withColumn(f.name, F.col(f.name).cast("int"))
            # strings need no cast
        df.write.mode("overwrite").format("parquet").saveAsTable(t)
        print(f"Loaded {t} from {path}")

```

WARNING: Using incubator modules: jdk.incubator.vector

Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties

26/01/07 20:47:55 WARN Utils: Your hostname, kinchins-MacBook-Pro.local, resolves to a loopback address: 127.0.0.1; using 192.168.4.184 instead (on interface en0)

26/01/07 20:47:55 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address

Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties

Setting default log level to "WARN".

To adjust logging level use `sc.setLogLevel(newLevel)`. For SparkR, use `setLogLevel(newLevel)`.

26/01/07 20:47:56 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable

26/01/07 20:47:56 WARN Utils: Service 'SparkUI' could not bind on port 4040.

Attempting port 4041.

Spark version: 4.0.1

```
[2]: DB = "tpch"                                # change as you like
      OUTPUT="customer_policy_results" # output directory for results

# Define canonical TPC-H schemas ----
dec = lambda p, s: T.DecimalType(p, s)
SCHEMAS = {
    "region": T.StructType([
        T.StructField("r_regionkey", T.IntegerType(), False),
        T.StructField("r_name", T.StringType(), False),
        T.StructField("r_comment", T.StringType(), True),
    ]),
    "nation": T.StructType([
        T.StructField("n_nationkey", T.IntegerType(), False),
        T.StructField("n_name", T.StringType(), False),
        T.StructField("n_regionkey", T.IntegerType(), False),
        T.StructField("n_comment", T.StringType(), True),
    ]),
    "part": T.StructType([
        T.StructField("p_partkey", T.IntegerType(), False),
        T.StructField("p_name", T.StringType(), False),
        T.StructField("p_mfgr", T.StringType(), False),
        T.StructField("p_brand", T.StringType(), False),
        T.StructField("p_type", T.StringType(), False),
        T.StructField("p_size", T.IntegerType(), False),
        T.StructField("p_container", T.StringType(), False),
        T.StructField("p_retailprice", T.FloatType(), False),
        T.StructField("p_comment", T.StringType(), True),
    ]),
    "supplier": T.StructType([
        T.StructField("s_suppkey", T.IntegerType(), False),
        T.StructField("s_name", T.StringType(), False),
        T.StructField("s_address", T.StringType(), False),
        T.StructField("s_nationkey", T.IntegerType(), False),
        T.StructField("s_phone", T.StringType(), False),
        T.StructField("s_acctbal", T.FloatType(), False),
        T.StructField("s_comment", T.StringType(), True),
    ]),
    "partsupp": T.StructType([
        T.StructField("ps_partkey", T.IntegerType(), False),
        T.StructField("ps_suppkey", T.IntegerType(), False),
        T.StructField("ps_availqty", T.IntegerType(), False),
        T.StructField("ps_supplycost", T.FloatType(), False),
        T.StructField("ps_comment", T.StringType(), True),
    ]),
}
```

```

"customer": T.StructType([
    T.StructField("c_custkey", T.IntegerType(), False),
    T.StructField("c_name", T.StringType(), False),
    T.StructField("c_address", T.StringType(), False),
    T.StructField("c_nationkey", T.IntegerType(), False),
    T.StructField("c_phone", T.StringType(), False),
    T.StructField("c_acctbal", T.FloatType(), False),
    T.StructField("c_mktsegment", T.StringType(), True),
    T.StructField("c_comment", T.StringType(), True),
]),
"orders": T.StructType([
    T.StructField("o_orderkey", T.IntegerType(), False),
    T.StructField("o_custkey", T.IntegerType(), False),
    T.StructField("o_orderstatus", T.StringType(), False),
    T.StructField("o_totalprice", T.FloatType(), False),
    T.StructField("o_orderdate", T.DateType(), False),
    T.StructField("o_orderpriority", T.StringType(), False),
    T.StructField("o_clerk", T.StringType(), False),
    T.StructField("o_shippriority", T.IntegerType(), False),
    T.StructField("o_comment", T.StringType(), True),
]),
"lineitem": T.StructType([
    T.StructField("l_orderkey", T.IntegerType(), False),
    T.StructField("l_partkey", T.IntegerType(), False),
    T.StructField("l_suppkey", T.IntegerType(), False),
    T.StructField("l_linenum", T.IntegerType(), False),
    T.StructField("l_quantity", T.FloatType(), False),
    T.StructField("l_extendedprice", T.FloatType(), False),
    T.StructField("l_discount", T.FloatType(), False),
    T.StructField("l_tax", T.FloatType(), False),
    T.StructField("l_returnflag", T.StringType(), False),
    T.StructField("l_linestatus", T.StringType(), False),
    T.StructField("l_shipdate", T.DateType(), False),
    T.StructField("l_commitdate", T.DateType(), False),
    T.StructField("l_receiptdate", T.DateType(), False),
    T.StructField("l_shipinstruct", T.StringType(), False),
    T.StructField("l_shipmode", T.StringType(), False),
    T.StructField("l_comment", T.StringType(), True),
]),
}
TABLES = {
    "region", "nation", "part", "supplier", "partsupp", "customer", "orders", "lineitem"
}
spark.sql(f"CREATE DATABASE IF NOT EXISTS {DB}")
spark.catalog.setCurrentDatabase(DB)

create_empty_tables()

```

Created empty schema in database `tpch`.

```
[ ]: SRC_CSV_DIR = "./raw"      # where *.tbl live
load_from_csv(SRC_CSV_DIR)

# Quick smoke test:
spark.sql("SHOW TABLES").show(truncate=False)
# show record count for each table
for t in TABLES:
    print(f"Table {t} has {spark.sql(f'SELECT count(*) FROM {t}').
    ↪collect()[0][0]} records.")
```

1.1 TPC-H Queries Evaluated

- **Q1 - Pricing Summary Report Query** This query reports the amount of business that was billed, shipped, and returned.
- **Q4 - Order Priority Checking Query** This query determines how well the order priority system is working and gives an assessment of customer satisfaction.
- **Q13 - Customer Distribution Query** This query seeks relationships between customers and the size of their orders.
- **Q16 - Parts/Supplier Relationship Query** This query finds out how many suppliers can supply parts with given attributes. It might be used, for example, to determine whether there is a sufficient number of suppliers for heavily ordered parts.

1.2 PrivateSQL paper evaluation

1.2.1 Dataset: TPC-H

1.2.2 Privacy Policy: Customer

1.2.3 Privacy Budget : 1.0

privacy model: customer table is private orders table have foreign key of customer table (custkey)
lineitem table have foreign key of orders table (orderkey)

```
[3]: SRC_CSV_DIR = "./raw"
dataframes = {}
tpch_query_results = {}

TABLES = [
    ↪["region", "nation", "part", "supplier", "partsupp", "customer", "orders", "lineitem"]
for table_name in TABLES:
    path = f"{SRC_CSV_DIR}/{table_name}.tbl"
    schema = SCHEMAS[table_name]
    dataframes[table_name] = (spark.read
        .option("sep", "|")
        .option("header", "false")
        .option("mode", "PERMISSIVE")
```

```

        .schema(schema)
        .csv(path))

#needed for q1
lineitem_with_disc_view = (dataframes["lineitem"]
    .withColumn("disc_price",
        (dataframes["lineitem"]["l_extendedprice"].cast("double") * (1.
↪0 - dataframes["lineitem"]["l_discount"].cast("double"))))
    .withColumn("charge",
        (dataframes["lineitem"]["l_extendedprice"].cast("double") * (1.
↪0 - dataframes["lineitem"]["l_discount"].cast("double")) * (1.0 + ↪
↪dataframes["lineitem"]["l_tax"].cast("double"))))

)
dataframes["lineitem_with_disc"] = lineitem_with_disc_view

for table_name in dataframes:
    print(f"Schema for table: {table_name}")
    dataframes[table_name].show(5)

budget = PureDPBudget(float("inf"))
#budget = PureDPBudget(3)

id_space = "customer_policy_id_space"

session = (
    Session.Builder()
    .with_privacy_budget(budget)
    .with_id_space(id_space)
    .with_private_dataframe(
        "lineitem_with_disc",
        dataframes["lineitem_with_disc"],
        protected_change=AddMaxRows(287),
    )
    .with_private_dataframe(
        "lineitem",
        dataframes["lineitem"],
        protected_change=AddMaxRows(287),
    )
    .with_private_dataframe(
        "orders",
        dataframes["orders"],
        protected_change=AddMaxRows(41),
    )
    .with_private_dataframe(

```

```

        "customer",
        dataframes["customer"],
        protected_change=AddOneRow(),
    )
    .with_public_dataframe(
        "region",
        dataframes["region"],
    )
    .with_public_dataframe(
        "nation",
        dataframes["nation"],
    )
    .with_public_dataframe(
        "part",
        dataframes["part"],
    )
    .with_public_dataframe(
        "supplier",
        dataframes["supplier"],
    )
    .with_public_dataframe(
        "partsupp",
        dataframes["partsupp"],
    )
    .build()
)

```

Schema for table: region

+-----+-----+-----+		
r_regionkey	r_name	r_comment
+-----+-----+-----+		
0	AFRICA	lar deposits. bli...
1	AMERICA	hs use ironic, ev...
2	ASIA	ges. thinly even ...
3	EUROPE	ly final courts c...
4 MIDDLE EAST		uickly special ac...
+-----+-----+-----+		

Schema for table: nation

+-----+-----+-----+-----+			
n_nationkey	n_name	n_regionkey	n_comment
+-----+-----+-----+-----+			
0	ALGERIA	0	haggle. carefull...
1	ARGENTINA	1	al foxes promise ...
2	BRAZIL	1	y alongside of th...
3	CANADA	1	eas hang ironic, ...
4	EGYPT	4	y above the caref...
+-----+-----+-----+-----+			

only showing top 5 rows

Schema for table: part

p_partkey	p_name	p_mfgr	p_brand	p_type	p_size	p_container	p_retailprice	p_comment
7	JUMBO PKG	901.0	ly. slyly ironi					
1	LG CASE	902.0	lar accounts amo					
21	WRAP CASE	903.0	egular deposits hag					
14	MED DRUM	904.0	p furiously r					
15	SM PKG	905.0	wake carefully					

only showing top 5 rows

Schema for table: supplier

s_suppkey	s_name	s_address	s_nationkey	s_phone	s_acctbal	s_comment
1	Supplier#000000001	N kD4on90M Ipw3,...	17	27-918-335-1736	5755.94	each slyly above ...
2	Supplier#000000002	89eJ5ksX3ImxJQBvx...	5	15-679-861-2259	4032.68	slyly bold instr...
3	Supplier#000000003	q1,G3Pj60jIuUYfUo...	1	11-383-516-1199	4192.4	blithely silent r...
4	Supplier#000000004	Bk7ah4CK8SYQTepEm...	15	25-843-787-7479	4641.08	riously even requ...
5	Supplier#000000005	Gcdm2rJRz15qlTVzc	11	21-151-690-3663	-283.84	. slyly regular p...

only showing top 5 rows

Schema for table: partsupp

ps_partkey	ps_suppkey	ps_availqty	ps_supplycost	ps_comment
1	2	3325	771.64	, even theodolite...
1	2502	8076	993.49	ven ideas. quickl...
1	5002	3956	337.09	after the fluffil...

1	7502	4069	357.84	al, regular depen...
2	3	8895	378.49	nic accounts. fin...

only showing top 5 rows

Schema for table: customer

c_custkey	c_name	c_address	c_nationkey	c_phone	c_acctbal	c_mktsegment	c_comment
1	Customer#000000001	IVhzIApeRb ot,c,E	15	25-989-741-2988	711.56	BUILDING	to the even, regu...
2	Customer#000000002	XSTf4,NCwDVaWNe6t...	13	23-768-687-3665	121.65	AUTOMOBILE	l accounts. blith...
3	Customer#000000003	MG9kdTD2WBHm	1	11-719-748-3364	7498.12	AUTOMOBILE	deposits eat sly...
4	Customer#000000004	XxVSJsLAGtn	4	14-128-190-5944	2866.83	MACHINERY	requests. final,...
5	Customer#000000005	KvpyuHCplrB84WgAi...	3	13-750-942-6364	794.47	HOUSEHOLD	n accounts will h...

only showing top 5 rows

Schema for table: orders

o_orderkey	o_custkey	o_orderstatus	o_totalprice	o_orderdate	o_orderpriority	o_clerk	o_shippriority	o_comment
1	36901	0	173665.47	1996-01-02	5-LOW	Clerk#000000951	0	nstructions sleep...
2	78002	0	46929.18	1996-12-01	1-URGENT	Clerk#000000880	0	foxes. pending a...
3	123314	F	193846.25	1993-10-14	5-LOW	Clerk#000000955	0	sly final account...
4	136777	0	32151.78	1995-10-11	5-LOW	Clerk#000000124	0	sits. slyly regul...
5	44485	F	144659.2	1994-07-30	5-LOW	Clerk#000000925	0	quickly. bold dep...

only showing top 5 rows

Schema for table: lineitem

l_orderkey	l_partsuppkey	l_partkey	l_suppkey	l_quantity	l_warehouse	l_address	l_comment
------------	---------------	-----------	-----------	------------	-------------	-----------	-----------

l_orderkey	l_partkey	l_suppkey	l_linenum	l_quantity	l_extendedprice	l_discount	l_tax	l_returnflag	l_linestatus	l_shipdate	l_commitdate	l_receiptdate	l_shipinstruct	l_shipmode	l_comment
1	155190	7706	1	17.0	21168.23	0.04	0.02	N	0	1996-03-13	1996-02-12	1996-03-22	DELIVER IN PERSON	TRUCK	egular courts abo...
1	67310	7311	2	36.0	45983.16	0.09	0.06	N	0	1996-04-12	1996-02-28	1996-04-20	TAKE BACK RETURN	MAIL	ly final dependen...
1	63700	3701	3	8.0	13309.6	0.1	0.02	N	0	1996-01-29	1996-03-05	1996-01-31	TAKE BACK RETURN	REG AIR	riously. regular,...
1	2132	4633	4	28.0	28955.64	0.09	0.06	N	0	1996-04-21	1996-03-30	1996-05-16	NONE	AIR	lites. fluffily e...
1	24027	1534	5	24.0	22824.48	0.1	0.04	N	0	1996-03-30	1996-03-14	1996-04-01	NONE	FOB	pending foxes. s...

only showing top 5 rows

Schema for table: lineitem_with_disc

l_orderkey	l_partkey	l_suppkey	l_linenum	l_quantity	l_extendedprice	l_discount	l_tax	l_returnflag	l_linestatus	l_shipdate	l_commitdate	l_receiptdate	l_shipinstruct	l_shipmode	l_comment	disc_price	charge
1	155190	7706	1	17.0	21168.23	0.04	0.02	N	0	1996-03-13	1996-02-12	1996-03-22	DELIVER IN PERSON	TRUCK	egular courts abo...	20321.501268925873	20727.931285219973
1	67310	7311	2	36.0	45983.16	0.09	0.06	N	0	1996-04-12	1996-02-28	1996-04-20	TAKE BACK RETURN	MAIL	ly final dependen...	41844.675577738904	44355.356056285156
1	63700	3701	3	8.0	13309.6	0.1	0.02	N	0	1996-01-29	1996-03-05	1996-01-31	TAKE				

```

BACK RETURN|    REG AIR|riously.
regular,...|11978.639628604651|12218.212415821876|
|          1|          2132|          4633|          4|          28.0|          28955.64|
0.09| 0.06|          N|          0|1996-04-21| 1996-03-30| 1996-05-16|
NONE|          AIR|lites. fluffily e...| 26349.63286519656| 27930.61080177074|
|          1|          24027|          1534|          5|          24.0|          22824.48|
0.1| 0.04|          N|          0|1996-03-30| 1996-03-14| 1996-04-01|
NONE|          FOB| pending foxes. s...|20542.032387863874| 21363.71366501242|
+-----+-----+-----+-----+-----+-----+-----+
--+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+
+
only showing top 5 rows

```

```

[4]: def query_dp_eval(eps: PureDPBudget, session: Session, query_num, query_dp_run,
    num_runs: int = 1, start_run: int = 0):
    for run in range(start_run, num_runs):
        print(f"Running Q1 DP run {run+1}/{num_runs} with epsilon={eps.epsilon}...")
        runx = query_dp_run(eps, session)

        output_file = f"./output/{OUTPUT}/q{query_num}/
    q{query_num}_dp_result_eps_{eps.epsilon}_runs_{run+1}_csv"

        runx.coalesce(1).write \
            .mode("overwrite") \
            .option("header", "true") \
            .csv(output_file)

```

1.2.4 Q1 — Pricing Summary Report

```

SELECT
    l_returnflag,
    l_linestatus,
    SUM(l_quantity) AS sum_qty,
    SUM(l_extendedprice) AS sum_base_price,
    SUM(l_extendedprice * (1 - l_discount)) AS sum_disc_price,
    SUM(l_extendedprice * (1 - l_discount) * (1 + l_tax)) AS sum_charge,
    AVG(l_quantity) AS avg_qty,
    AVG(l_extendedprice) AS avg_price,
    AVG(l_discount) AS avg_disc,
    COUNT(_) AS count_order
FROM lineitem
WHERE l_shipdate <= DATE '1998-12-01' - INTERVAL '[DELTA]' DAY
GROUP BY l_returnflag, l_linestatus
ORDER BY l_returnflag, l_linestatus;

```

```

[5]: def q1_dp_run(eps: PureDPBudget, session: Session):

    #1) Create V_q1 with casts + derived columns
    DELTA = 90

    q1_view = (
        QueryBuilder("lineitem_with_disc")
        .select([
            "l_returnflag",
            "l_linestatus",
            "l_quantity",
            "l_extendedprice",
            "l_discount",
            "l_tax",
            "l_shipdate",
            "disc_price",
            "charge",
        ])
    )

    # delete V_q1 only if it exists to avoid errors
    try:
        session.delete_view("V_q1")
    except Exception as e:
        pass

    session.create_view(source_id="V_q1", query_expr=q1_view, cache=True)

    #2) Group + filter (use Spark-friendly interval)
    groupby_keys = KeySet.from_dict({
        "l_returnflag": ["A", "N", "R"],
        "l_linestatus": ["F", "O"],
    })

    q1_linear_grouped = (
        QueryBuilder("V_q1")
        .filter(f"l_shipdate <= DATE '1998-12-01' - INTERVAL {DELTA} DAYS")
        .groupby(groupby_keys)
        .count(name="count_order")
    )

    dp_count_order = session.evaluate(q1_linear_grouped, privacy_budget=eps)
    return dp_count_order

```

1.2.5 Q4 - Order Priority Checking Query

```
SELECT
    o_orderpriority,
    COUNT(*) AS order_count
FROM
    orders
WHERE
    o_orderdate >= DATE '[DATE]'
    AND o_orderdate < DATE '[DATE]' + INTERVAL '3' MONTH
    AND EXISTS (
        SELECT
            *
        FROM
            lineitem
        WHERE
            l_orderkey = o_orderkey
            AND l_commitdate < l_receiptdate
    )
GROUP BY
    o_orderpriority
ORDER BY
    o_orderpriority;
```

```
[6]: def q4_dp_DropExcess_1_run(eps: PureDPBudget, session: Session):
    print("Building Q4 deduped view V_q4...")
    from tmlt.analytics.query_builder import QueryBuilder

    # lineitem filtered
    q_lineitem_filtered = (
        QueryBuilder("lineitem")
        .filter("l_commitdate < l_receiptdate")
        .select(["l_orderkey"])
        .rename({"l_orderkey": "orderkey"})
    )

    # orders columns (include orderdate so you DON'T need to join orders again,
    ↪ later)
    q_orders = (
        QueryBuilder("orders")
        .select(["o_orderkey", "o_orderpriority", "o_orderdate"])
        .rename({"o_orderkey": "orderkey"})
    )

    # join
    q4_view = q_orders.join_private(
        right_operand=q_lineitem_filtered,
        join_columns=["orderkey"],
```

```

        truncation_strategy_left=TruncationStrategy.DropExcess(1),
        truncation_strategy_right=TruncationStrategy.DropExcess(1),
    ).select(["orderkey", "o_orderpriority", "o_orderdate"])

    # create session view
    try:
        session.delete_view("V_q4")
    except Exception:
        pass

    session.create_view(source_id="V_q4", query_expr=q4_view, cache=True)

    start_date = "1993-07-01"
    eps = PureDPBudget(epsilon=1.0)

    groupby_keys = KeySet.from_dict({
        "o_orderpriority": ["1-URGENT", "2-HIGH", "3-MEDIUM", "4-NOT_
↪SPECIFIED", "5-LOW"]
    })

    q_orders_2 = (
        QueryBuilder("orders")
        .select(["o_orderkey", "o_orderdate"])
        .filter(
            f"o_orderdate >= DATE '{start_date}' AND "
            f"o_orderdate < DATE '{start_date}' + INTERVAL 3 MONTH"
        )
        .rename({"o_orderkey": "orderkey"})
    )

    q4_1 = (
        QueryBuilder("V_q4").join_private(
            right_operand=q_orders_2,
            join_columns=["orderkey"],
            truncation_strategy_left=TruncationStrategy.DropExcess(1),
            truncation_strategy_right=TruncationStrategy.DropExcess(1),
        )
        .select(["o_orderpriority"])
        .groupby(groupby_keys)
    )

    q4_count = q4_1.count(name="order_count")

    q4_dp_result = session.evaluate(q4_count, privacy_budget=eps)
    ##q4_dp_result.orderBy("o_orderpriority").show(truncate=False)
    #q4_dp_result.show(truncate=False)

    return q4_dp_result

```

```

[7]: def q4_dp_DropNonUnique_run(eps: PureDPBudget, session: Session):
    print("Building Q4 deduped view V_q4...")
    from tmlt.analytics.query_builder import QueryBuilder

    # lineitem filtered
    q_lineitem_filtered = (
        QueryBuilder("lineitem")
        .filter("l_commitdate < l_receiptdate")
        .select(["l_orderkey"])
        .rename({"l_orderkey": "orderkey"})
    )

    # orders columns (include orderdate so you DON'T need to join orders again,
    ↪ later)
    q_orders = (
        QueryBuilder("orders")
        .select(["o_orderkey", "o_orderpriority", "o_orderdate"])
        .rename({"o_orderkey": "orderkey"})
    )

    # join
    q4_view = q_orders.join_private(
        right_operand=q_lineitem_filtered,
        join_columns=["orderkey"],
        truncation_strategy_left=TruncationStrategy.DropNonUnique(),
        truncation_strategy_right=TruncationStrategy.DropNonUnique(),
    ).select(["orderkey", "o_orderpriority", "o_orderdate"])

    # create session view
    try:
        session.delete_view("V_q4")
    except Exception:
        pass

    session.create_view(source_id="V_q4", query_expr=q4_view, cache=True)

    start_date = "1993-07-01"
    eps = PureDPBudget(epsilon=1.0)

    groupby_keys = KeySet.from_dict({
        "o_orderpriority": ["1-URGENT", "2-HIGH", "3-MEDIUM", "4-NOT_
    ↪ SPECIFIED", "5-LOW"]
    })

    q_orders_2 = (
        QueryBuilder("orders")
        .select(["o_orderkey", "o_orderdate"])

```

```

        .filter(
            f"o_orderdate >= DATE '{start_date}' AND "
            f"o_orderdate < DATE '{start_date}' + INTERVAL 3 MONTH"
        )
        .rename({"o_orderkey": "orderkey"})
    )
    q4_1 = (
        QueryBuilder("V_q4").join_private(
            right_operand=q_orders_2,
            join_columns=["orderkey"],
            truncation_strategy_left=TruncationStrategy.DropNonUnique(),
            truncation_strategy_right=TruncationStrategy.DropNonUnique(),
        )
        .select(["o_orderpriority"])
        .groupby(groupby_keys)
    )

    q4_count = q4_1.count(name="order_count")

    q4_dp_result = session.evaluate(q4_count, privacy_budget=eps)
    ##q4_dp_result.orderBy("o_orderpriority").show(truncate=False)
    #q4_dp_result.show(truncate=False)

    return q4_dp_result

```

1.2.6 Q13 - Customer Distribution Query

```

SELECT
    c_count,
    COUNT(*) AS custdist
FROM (
    SELECT
        c_custkey,
        COUNT(o_orderkey) AS c_count
    FROM
        customer
    LEFT OUTER JOIN orders
        ON c_custkey = o_custkey
        AND o_comment NOT LIKE '%[WORD1]%[WORD2]%'
    GROUP BY
        c_custkey
) AS c_orders (c_custkey, c_count)
GROUP BY
    c_count
ORDER BY
    custdist DESC,

```

```
c_count DESC;
```

```
[8]: def q13_dp_DropExcess_1_run(eps: PureDPBudget, session: Session):

    WORD1 = 'special'
    WORD2 = 'requests'

    """
    SELECT
    c.c_custkey,
    COUNT(o.o_orderkey) AS c_count
    FROM customer c
    LEFT JOIN orders o
    ON o.o_custkey = c.c_custkey
    AND o_comment NOT LIKE '%{WORD1}%{WORD2}%'
    GROUP BY c.c_custkey;
    """

    q13_view_1 = (
        QueryBuilder("customer")
        .select(["c_custkey"])
        .rename({"c_custkey": "custkey"})
    )

    q13_view_2 = (
        QueryBuilder("orders")
        .select(["o_orderkey", "o_custkey", "o_comment"])
        .filter(f"o_comment NOT LIKE '%{WORD1}%{WORD2}%'")
        .rename({"o_custkey": "custkey"})
    )

    q13_view_3 = q13_view_1.join_private(
        q13_view_2,
        join_columns=["custkey"],
        truncation_strategy_left=TruncationStrategy.DropExcess(1),
        truncation_strategy_right=TruncationStrategy.DropExcess(1),
    )

    try:
        session.delete_view("V_q13")
    except Exception:
        pass

    session.create_view(source_id="V_q13", query_expr=q13_view_3, cache=True)

    custkey_groups = list(range(1, 1000000))
    custkey_group_keyset = KeySet.from_dict({"custkey": custkey_groups})
```

```

q13_1 = (
    QueryBuilder("V_q13")
    .groupby(custkey_group_keyset)
    .count(name="c_count")
)
x = session.evaluate(q13_1, privacy_budget=eps)
x.show(truncate=False)

count_groups = list(range(1, 10000))
count_group_keyset = KeySet.from_dict({"c_count": count_groups})
print(type(x))
dp_result = (
    x.select(["custkey", "c_count"])
    .groupby(["c_count"])
    .agg(F.count("*").alias("custdist"))
)

return dp_result

```

```
[9]: def q13_dp_DropNonUnique_run(eps: PureDPBudget, session: Session):
```

```

WORD1 = 'special'
WORD2 = 'requests'

"""
SELECT
c.c_custkey,
COUNT(o.o_orderkey) AS c_count
FROM customer c
LEFT JOIN orders o
ON o.o_custkey = c.c_custkey
AND o_comment NOT LIKE '%{WORD1}%{WORD2}%'
GROUP BY c.c_custkey;
"""

q13_view_1 = (
    QueryBuilder("customer")
    .select(["c_custkey"])
    .rename({"c_custkey": "custkey"})
)

q13_view_2 = (
    QueryBuilder("orders")
    .select(["o_orderkey", "o_custkey", "o_comment"])
    .filter(f"o_comment NOT LIKE '%{WORD1}%{WORD2}%'")
)

```

```

        .rename({"o_custkey": "custkey"})
    )

    q13_view_3 = q13_view_1.join_private(
        q13_view_2,
        join_columns=["custkey"],
        truncation_strategy_left=TruncationStrategy.DropNonUnique(),
        truncation_strategy_right=TruncationStrategy.DropNonUnique(),
    )

    try:
        session.delete_view("V_q13")
    except Exception:
        pass

    session.create_view(source_id="V_q13", query_expr=q13_view_3, cache=True)

    custkey_groups = list(range(1, 1000000))
    custkey_group_keyset = KeySet.from_dict({"custkey": custkey_groups})

    q13_1 = (
        QueryBuilder("V_q13")
        .groupby(custkey_group_keyset)
        .count(name="c_count")
    )
    x = session.evaluate(q13_1, privacy_budget=eps)
    x.show(truncate=False)

    count_groups = list(range(1, 10000))
    count_group_keyset = KeySet.from_dict({"c_count": count_groups})
    print(type(x))
    dp_result = (
        x.select(["custkey", "c_count"])
        .groupby(["c_count"])
        .agg(F.count("*").alias("custdist"))
    )

    return dp_result

```

```

[ ]: for run in [1000]:
    eps = PureDPBudget(1)
    query_dp_eval(eps, session, query_num="1", query_dp_run=q1_dp_run,
    ↪ num_runs=run)
    query_dp_eval(eps, session, query_num="4_dp_DropExcess_1",
    ↪ query_dp_run=q4_dp_DropExcess_1_run, num_runs=run)

```

```

    query_dp_eval(eps, session, query_num="4_dp_DropNonUnique",
    ↪query_dp_run=q4_dp_DropNonUnique_run, num_runs=run)
    query_dp_eval(eps, session, query_num="13_dp_DropExcess_1",
    ↪query_dp_run=q13_dp_DropExcess_1_run, num_runs=run)
    query_dp_eval(eps, session, query_num="13_dp_DropNonUnique_run",
    ↪query_dp_run=q13_dp_DropNonUnique_run, num_runs=run)

```

1.3 Calculate Error

```

[10]: from typing import List

def compute_errors_privatesql_style(
    dp_df,
    truth_df,
    keys: List[str],
    metric: str = "count_order",
    run_col: str = "run_id",
    join_type: str = "left",
    truth_missing_as_zero: bool = True,
):
    """
    Compute PrivateSQL-style errors per output group per run.
    Qerror(y, y~) = |y - y~|
    RelError(y, y~) = |y - y~| / max(50, y)

    Returns columns:
        run_id,
        keys,
        <metric>_truth,
        <metric>_dp,
        qerror_<metric>,
        relerror_<metric>
    """

    # Prepare DP side
    dp = (
        dp_df
        .withColumn(metric, F.col(metric).cast("double"))
        .withColumnRenamed(metric, f"{metric}_dp")
    )
    #print("DP side:")
    #dp.show()

    # Prepare truth side

```

```

tr = (
    truth_df
    .withColumn(metric, F.col(metric).cast("double"))
    .withColumnRenamed(metric, f"{metric}_truth")
)
#print("Truth side:")
#tr.show()

joined = dp.alias("dp").join(tr.alias("tr"), on=keys, how=join_type)
#print("Joined:")
#joined.show()

if truth_missing_as_zero:
    joined = joined.withColumn(
        f"{metric}_truth",
        F.coalesce(F.col(f"{metric}_truth"), F.lit(0.0))
    )

# Absolute error |y - y~|
joined = joined.withColumn(
    f"qerror",
    F.abs(F.col(f"{metric}_truth") - F.col(f"{metric}_dp"))
)

# Relative error |y - y~| / max(50, y)
joined = joined.withColumn(
    f"relerror",
    F.col(f"qerror") /
    F.greatest(F.lit(50.0), F.col(f"{metric}_truth"))
)

return joined.select(
    run_col,
    *keys,
    f"{metric}_truth",
    f"{metric}_dp",
    f"qerror",
    f"relerror",
)

```

[11]: #Q1

```

# -----
# CONFIG
# -----
keys = ["l_returnflag", "l_linestatus"]
metrics = ["count_order"]

```

```

metric_casts = {"count_order": "long"} # important: count is integer

eps_val = 1
truth_path = "./output/no_dp/q1_result_csv"

# -----
# LOAD TRUTH
# -----
truth = (spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .csv(truth_path)
    .select(*keys, "count_order")
    .withColumn("count_order", F.col("count_order").cast("long"))
)

# -----
# LOAD RUNS (ALL 1000)
# -----
runs_glob = f"./output/{OUTPUT}/q1/q1_dp_result_eps_{eps_val}_runs_*_csv"

dp_runs = (spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .csv(runs_glob)
    .withColumn(
        "run_id",
        F.regexp_extract(F.input_file_name(), r"runs_(\d+)_csv", 1).cast("int")
    )
    .select("run_id", *keys, "count_order")
    .withColumn("count_order", F.col("count_order").cast("long"))
)
print("Q1")
print("Distinct runs =", dp_runs.select("run_id").distinct().count())

err_df = compute_errors_privatesql_style(
    dp_df=dp_runs,
    truth_df=truth,
    keys=keys,
    metric="count_order",
    run_col="run_id",
    join_type="left",
    truth_missing_as_zero=True,
)

per_run = err_df.groupBy("run_id").agg(
    F.avg("qerror").alias("avg_qerror_per_run"),

```

```

    F.avg("reerror").alias("avg_reerror_per_run"),
).orderBy("run_id")

reerror = (
    per_run
    .select("avg_reerror_per_run")
    .dropna()
    .toPandas()
)

qerror = (
    per_run
    .select("avg_qerror_per_run")
    .dropna()
    .toPandas()
)

overall = per_run.agg(
    F.avg("avg_qerror_per_run").alias("mean_qerror_over_runs"),
    F.avg("avg_reerror_per_run").alias("mean_reerror_over_runs"),
)

#overall.show(truncate=False)
tpch_query_results["Q1"] = {
    "mean_qerror_over_runs": float(overall.
    ↪collect()[0]["mean_qerror_over_runs"]),
    "mean_reerror_over_runs": float(overall.
    ↪collect()[0]["mean_reerror_over_runs"]),
    "qerror": qerror,
    "reerror": reerror
}

#overall.show(truncate=False)

plt.figure(figsize=(4, 4))
plt.boxplot(reerror, showfliers=False)
plt.xticks([1], ["Q1"])
plt.xlabel("TPC-H Query (Customer policy)")
plt.title("reerror Distribution (Q1)")
plt.tight_layout()
plt.show()

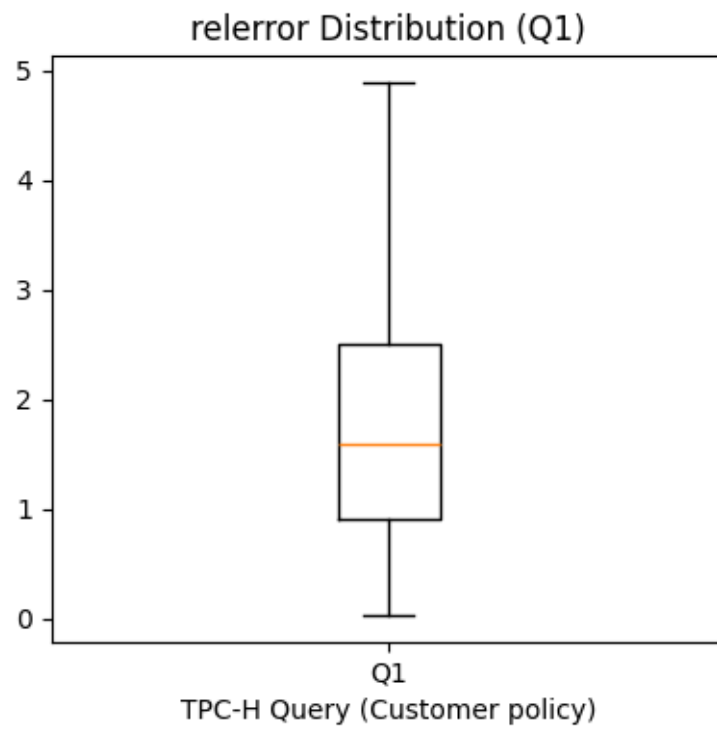
plt.figure(figsize=(4, 4))
plt.boxplot(qerror, showfliers=False)
plt.xticks([1], ["Q1"])
plt.xlabel("TPC-H Query (Customer policy)")

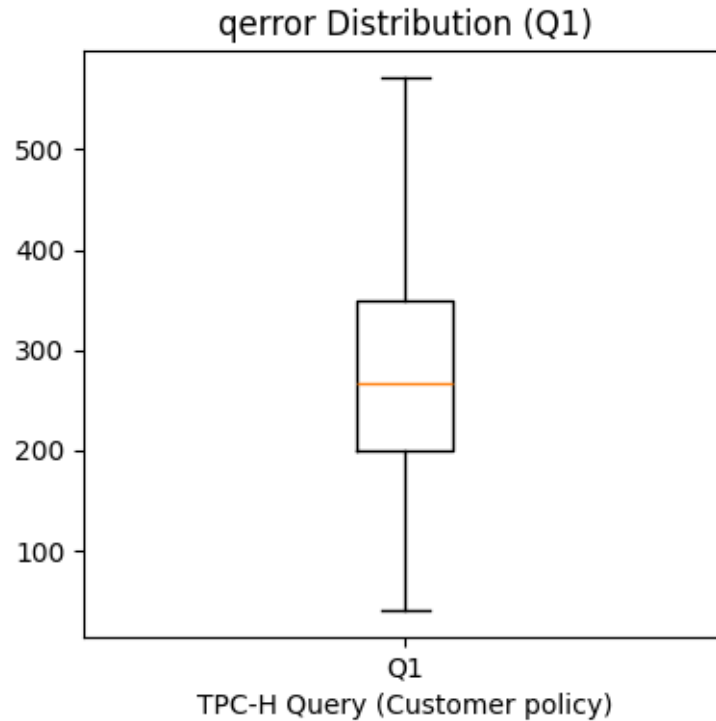
```

```
plt.title("qerror Distribution (Q1)")
plt.tight_layout()
plt.show()
```

Q1

Distinct runs = 1000





```
[12]: #Q4

# -----
# CONFIG
# -----
keys = ["o_orderpriority"]
metrics = "order_count"

eps_val = 1
truth_path = "./output/no_dp/q4_result_csv"
# -----
# LOAD TRUTH
# -----
truth = (spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .csv(truth_path)
    .select(*keys, "order_count")
    .withColumn("order_count", F.col("order_count").cast("long"))
)

# -----
# LOAD RUNS (ALL 1000)
```

```

# -----
test_case="q4_dp_DropExcess_1"
runs_glob = f"./output/{OUTPUT}/{test_case}/
↳{test_case}_dp_result_eps_{eps_val}_runs*_csv"

dp_runs = (spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .csv(runs_glob)
    .withColumn(
        "run_id",
        F.regexp_extract(F.input_file_name(), r"runs_(\d+)_csv", 1).cast("int")
    )
    .select("run_id", *keys, "order_count")
    .withColumn("order_count", F.col("order_count").cast("long")))
)

print(f"Q4 - {test_case}")
print("Distinct runs =", dp_runs.select("run_id").distinct().count())

err_df = compute_errors_privatesql_style(
    dp_df=dp_runs,
    truth_df=truth,
    keys=keys,
    metric=metrics,
    run_col="run_id",
    join_type="left",
    truth_missing_as_zero=True,
)

per_run = err_df.groupBy("run_id").agg(
    F.avg("qerror").alias("avg_qerror_per_run"),
    F.avg("relerror").alias("avg_relerror_per_run"),
).orderBy("run_id")

relerror = (
    per_run
    .select("avg_relerror_per_run")
    .dropna()
    .toPandas()
)

qerror = (
    per_run
    .select("avg_qerror_per_run")
    .dropna()
    .toPandas()
)

```

```

)

overall = per_run.agg(
    F.avg("avg_qerror_per_run").alias("mean_qerror_over_runs"),
    F.avg("avg_relerror_per_run").alias("mean_relerror_over_runs"),
)

tpch_query_results[test_case] = {
    "mean_qerror_over_runs": float(overall.
    ↪collect()[0]["mean_qerror_over_runs"]),
    "mean_relerror_over_runs": float(overall.
    ↪collect()[0]["mean_relerror_over_runs"]),
    "qerror": qerror,
    "relerror": relerror
}

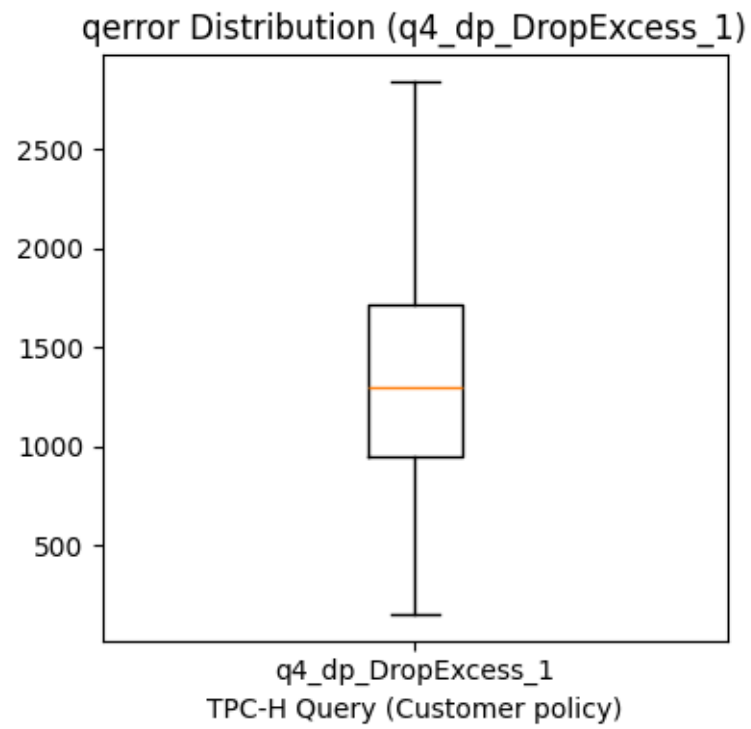
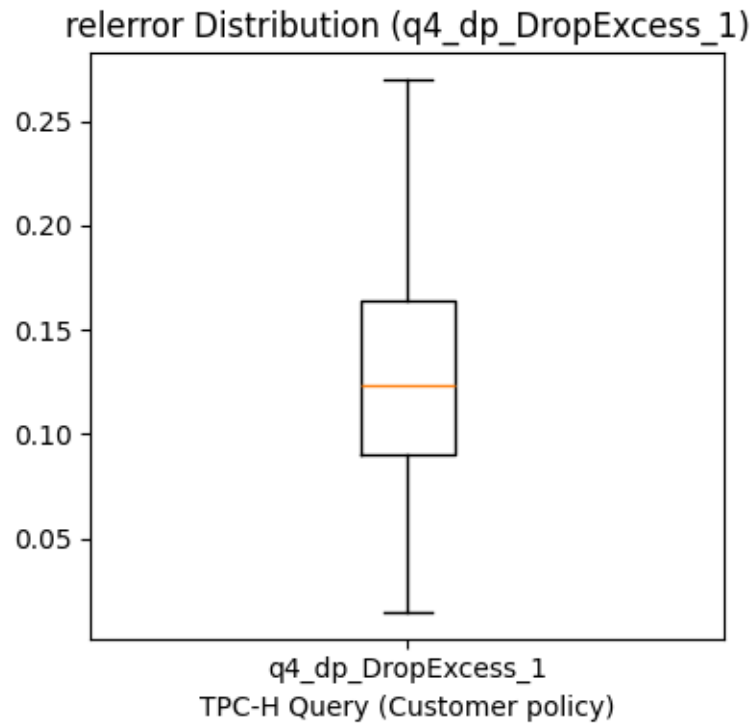
#overall.show(truncate=False)

plt.figure(figsize=(4, 4))
plt.boxplot(relerror, showfliers=False)
plt.xticks([1], [test_case])
plt.xlabel("TPC-H Query (Customer policy)")
plt.title("relerror Distribution " + f"({test_case})")
plt.tight_layout()
plt.show()

plt.figure(figsize=(4, 4))
plt.boxplot(qerror, showfliers=False)
plt.xticks([1], [test_case])
plt.xlabel("TPC-H Query (Customer policy)")
plt.title("qerror Distribution " + f"({test_case})")
plt.tight_layout()
plt.show()

```

Q4 - q4_dp_DropExcess_1
Distinct runs = 1000



```

[13]: #Q4

# -----
# CONFIG
# -----
keys = ["o_orderpriority"]
metrics = "order_count"

eps_val = 1
truth_path = "./output/no_dp/q4_result_csv"
# -----
# LOAD TRUTH
# -----
truth = (spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .csv(truth_path)
    .select(*keys, "order_count")
    .withColumn("order_count", F.col("order_count").cast("long")))
)

# -----
# LOAD RUNS (ALL 1000)
# -----
test_case="q4_dp_DropNonUnique"
runs_glob = f"./output/{OUTPUT}/{test_case}/
↳{test_case}_dp_result_eps_{eps_val}_runs*_csv"

dp_runs = (spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .csv(runs_glob)
    .withColumn(
        "run_id",
        F.regexp_extract(F.input_file_name(), r"runs_(\d+)_csv", 1).cast("int")
    )
    .select("run_id", *keys, "order_count")
    .withColumn("order_count", F.col("order_count").cast("long")))
)
print(f"Q4 - {test_case}")
print("Distinct runs =", dp_runs.select("run_id").distinct().count())

err_df = compute_errors_privatesql_style(
    dp_df=dp_runs,
    truth_df=truth,
    keys=keys,
    metric=metrics,

```

```

    run_col="run_id",
    join_type="left",
    truth_missing_as_zero=True,
)

#err_df.show(truncate=False)

per_run = err_df.groupBy("run_id").agg(
    F.avg("qerror").alias("avg_qerror_per_run"),
    F.avg("reerror").alias("avg_reerror_per_run"),
).orderBy("run_id")

reerror = (
    per_run
    .select("avg_reerror_per_run")
    .dropna()
    .toPandas()
)

qerror = (
    per_run
    .select("avg_qerror_per_run")
    .dropna()
    .toPandas()
)

overall = per_run.agg(
    F.avg("avg_qerror_per_run").alias("mean_qerror_over_runs"),
    F.avg("avg_reerror_per_run").alias("mean_reerror_over_runs"),
)

tpch_query_results[test_case] = {
    "mean_qerror_over_runs": float(overall.
    ↪collect()[0]["mean_qerror_over_runs"]),
    "mean_reerror_over_runs": float(overall.
    ↪collect()[0]["mean_reerror_over_runs"]),
    "qerror": qerror,
    "reerror": reerror
}

#overall.show(truncate=False)

plt.figure(figsize=(4, 4))
plt.boxplot(reerror, showfliers=False)
plt.xticks([1], [test_case])
plt.xlabel("TPC-H Query (Customer policy)")
plt.title("reerror Distribution " + f"({test_case})")

```

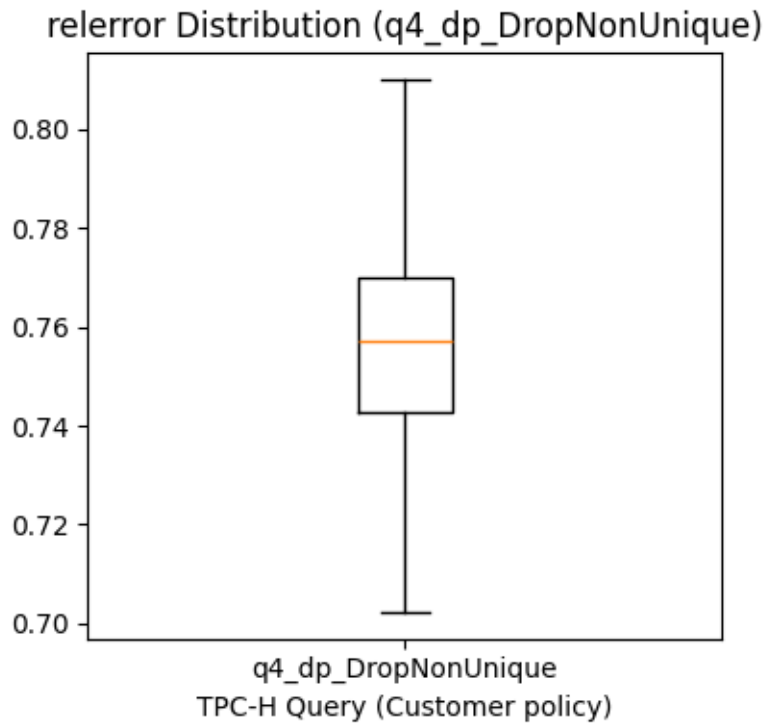
```

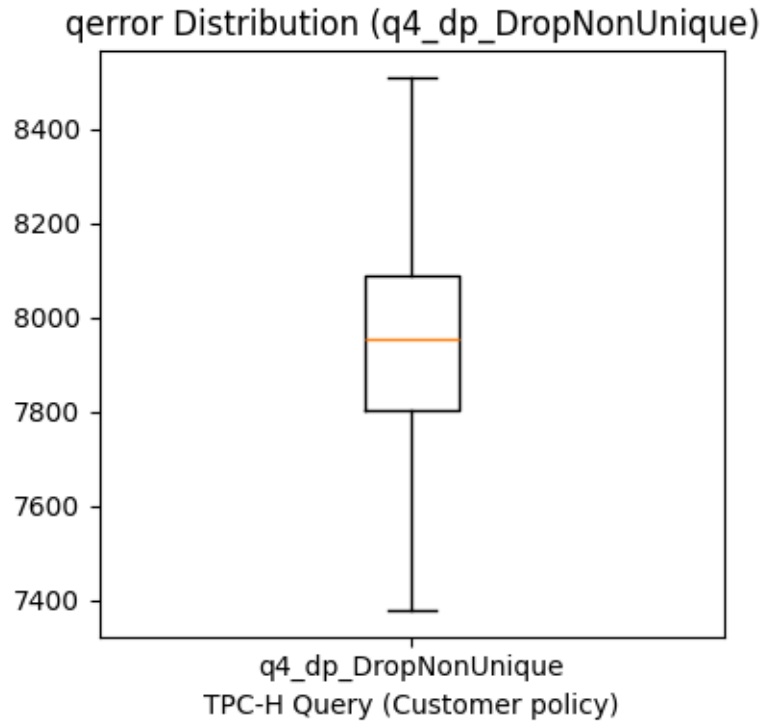
plt.tight_layout()
plt.show()

plt.figure(figsize=(4, 4))
plt.boxplot(qerror, showfliers=False)
plt.xticks([1], [test_case])
plt.xlabel("TPC-H Query (Customer policy)")
plt.title("qerror Distribution " + f"({test_case})")
plt.tight_layout()
plt.show()

```

Q4 - q4_dp_DropNonUnique
Distinct runs = 1000





```
[16]: #Q13

# -----
# CONFIG
# -----
keys = ["c_count"]
metrics = "custdist"

eps_val = 1
truth_path = "./output/no_dp/q13_result_csv"
# -----
# LOAD TRUTH
# -----
truth = (spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .csv(truth_path)
    .select(*keys, "custdist")
    .withColumn("custdist", F.col("custdist").cast("long"))
)

# -----
# LOAD RUNS (ALL 1000)
```

```

# -----
test_case="q13_dp_DropNonUnique_run"
runs_glob = f"./output/{OUTPUT}/{test_case}/
↳{test_case}_dp_result_eps_{eps_val}_runs*_csv"

dp_runs = (spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .csv(runs_glob)
    .withColumn(
        "run_id",
        F.regexp_extract(F.input_file_name(), r"runs_(\d+)_csv", 1).cast("int")
    )
    .select("run_id", *keys, "custdist")
    .withColumn("custdist", F.col("custdist").cast("long"))
)

print(f"Q13 - {test_case}")
print("Distinct runs =", dp_runs.select("run_id").distinct().count())

err_df = compute_errors_privatesql_style(
    dp_df=dp_runs,
    truth_df=truth,
    keys=keys,
    metric=metrics,
    run_col="run_id",
    join_type="left",
    truth_missing_as_zero=True,
)

#err_df.show(truncate=False)

per_run = err_df.groupBy("run_id").agg(
    F.avg("qerror").alias("avg_qerror_per_run"),
    F.avg("relererror").alias("avg_relererror_per_run"),
).orderBy("run_id")

relererror = (
    per_run
    .select("avg_relererror_per_run")
    .dropna()
    .toPandas()
)

qerror = (
    per_run
    .select("avg_qerror_per_run")

```

```

        .dropna()
        .toPandas()
    )

overall = per_run.agg(
    F.avg("avg_qerror_per_run").alias("mean_qerror_over_runs"),
    F.avg("avg_relerror_per_run").alias("mean_relerror_over_runs"),
)

tpch_query_results[test_case] = {
    "mean_qerror_over_runs": float(overall.
    ↪collect()[0]["mean_qerror_over_runs"]),
    "mean_relerror_over_runs": float(overall.
    ↪collect()[0]["mean_relerror_over_runs"]),
    "qerror": qerror,
    "relerror": relerror
}

#overall.show(truncate=False)

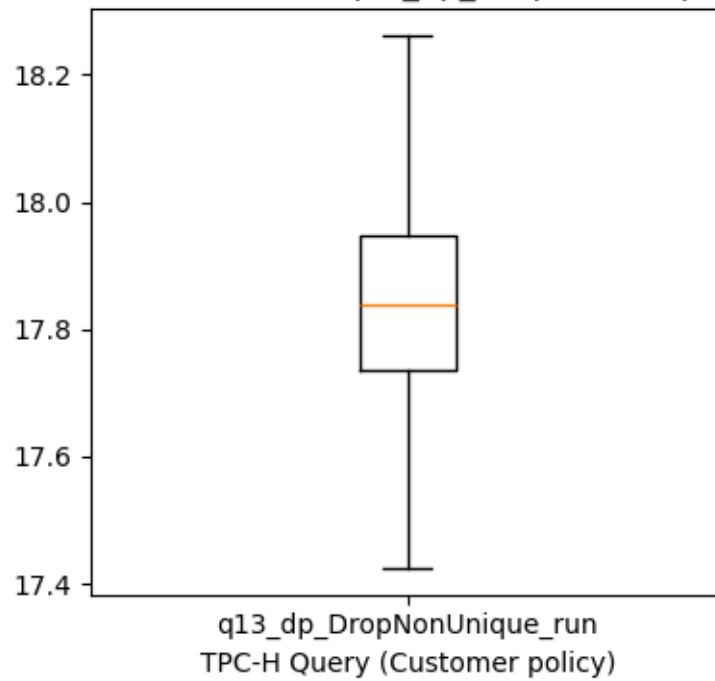
plt.figure(figsize=(4, 4))
plt.boxplot(relerror, showfliers=False)
plt.xticks([1], [test_case])
plt.xlabel("TPC-H Query (Customer policy)")
plt.title("relerror Distribution " + f"({test_case})")
plt.tight_layout()
plt.show()

plt.figure(figsize=(4, 4))
plt.boxplot(qerror, showfliers=False)
plt.xticks([1], [test_case])
plt.xlabel("TPC-H Query (Customer policy)")
plt.title("qerror Distribution " + f"({test_case})")
plt.tight_layout()
plt.show()

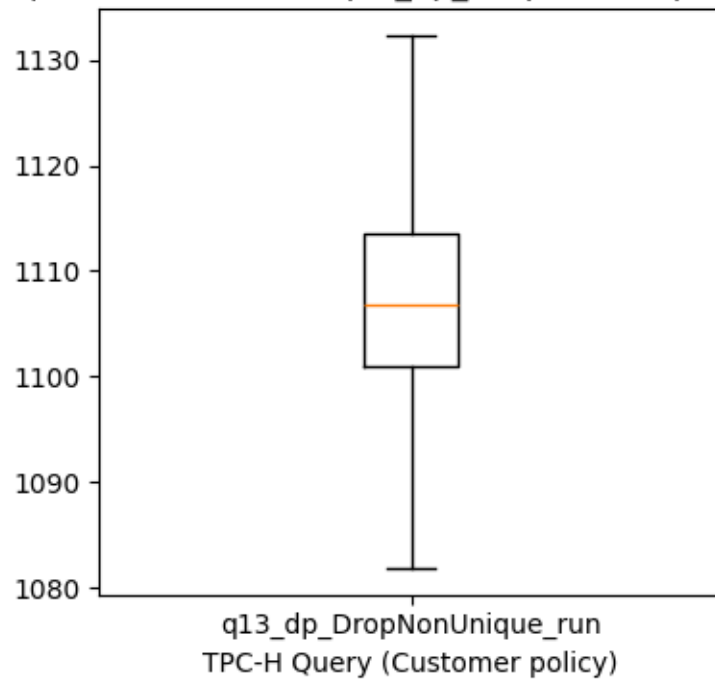
```

Q13 - q13_dp_DropNonUnique_run
 Distinct runs = 1000

relerror Distribution (q13_dp_DropNonUnique_run)



qerror Distribution (q13_dp_DropNonUnique_run)



```
[17]: print("opendp eval with TPC-H and customer policy Results Summary:")

import matplotlib.pyplot as plt

# Use a stable list of labels (convert to strings) and numeric positions for
# plotting
queries = list(tpch_query_results.keys())
queries_label = [str(q) for q in queries]
print("Queries evaluated:", queries_label)

avg_qerror = [tpch_query_results[q]["mean_qerror_over_runs"] for q in queries]
print("Average Qerror per query:", avg_qerror)
avg_relerror = [tpch_query_results[q]["mean_relerror_over_runs"] for q in
               queries]
print("Average RelError per query:", avg_relerror)

positions = list(range(len(queries_label)))

# Absolute error bar plot
fig, ax = plt.subplots(figsize=(4, 4))
ax.bar(positions, avg_qerror, color='tab:blue')
ax.set_xticks(positions)
ax.set_xticklabels(queries_label, rotation=45, ha='right')
ax.set_xlabel("TPC-H Query (Customer policy)")
ax.set_ylabel("Average Absolute Error (Qerror)")
ax.set_title("Average Absolute Error Across Runs")
plt.tight_layout()
plt.show()

# Relative error bar plot
fig2, ax2 = plt.subplots(figsize=(4, 4))
ax2.bar(positions, avg_relerror, color='tab:orange')
ax2.set_xticks(positions)
ax2.set_xticklabels(queries_label, rotation=45, ha='right')
ax2.set_xlabel("TPC-H Query (Customer policy)")
ax2.set_ylabel("Average Relative Error")
ax2.set_title("Average Relative Error Across Runs")
plt.tight_layout()
plt.show()

raw_qerror = [tpch_query_results[q]["qerror"] for q in queries]
raw_relerror = [tpch_query_results[q]["relerror"] for q in queries]
import numpy as np
raw_qerror_1d = [np.asarray(x).ravel() for x in raw_qerror]
```

```

plt.figure(figsize=(4, 4))
plt.boxplot(
    raw_qerror_1d,
    labels=queries_label,
    showfliers=False
)
plt.xticks(rotation=45, ha='right')
plt.xlabel("TPC-H Query (Customer policy)")
plt.ylabel("Average Absolute Error (Qerror)")
plt.title("qerror Distribution Across Runs")
plt.tight_layout()
plt.show()

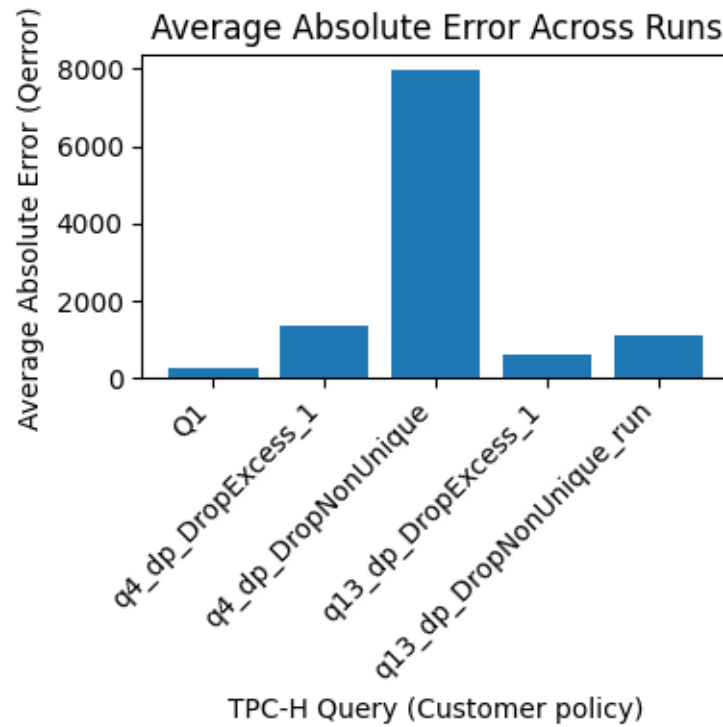
raw_relerror_1d = [np.asarray(x).ravel() for x in raw_relerror]

plt.figure(figsize=(4, 4))
plt.boxplot(
    raw_relerror_1d,
    labels=queries_label,
    showfliers=False
)
plt.xticks(rotation=45, ha='right')
plt.xlabel("TPC-H Query (Customer policy)")
plt.ylabel("Average Relative Error (RelError)")
plt.title("relerror Distribution Across Runs")
plt.tight_layout()
plt.show()

```

opendp eval with TPC-H and customer policy Results Summary:

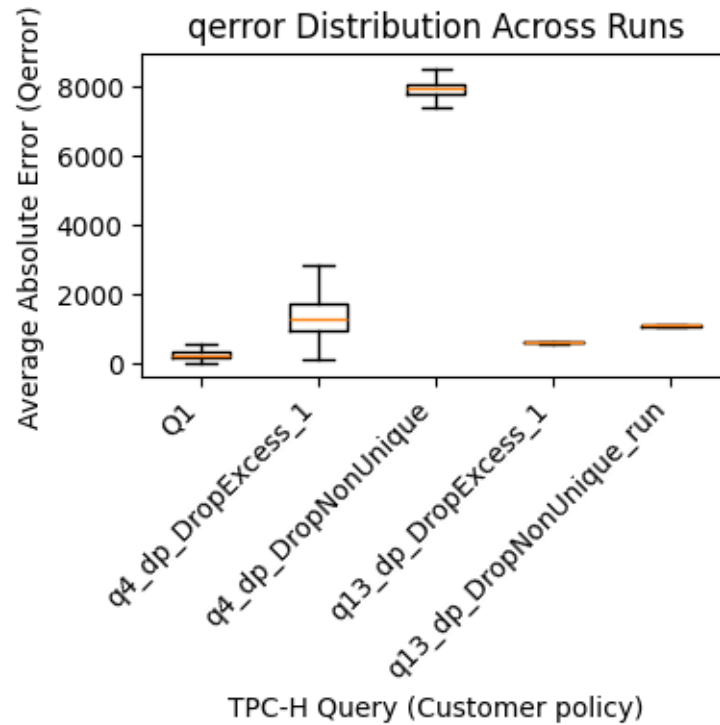
Queries evaluated: ['Q1', 'q4_dp_DropExcess_1', 'q4_dp_DropNonUnique',
'q13_dp_DropExcess_1', 'q13_dp_DropNonUnique_run']
Average Qerror per query: [282.6230000000001, 1374.2539999999992,
7951.9159999999984, 608.2021191311442, 1107.152784106976]
Average RelError per query: [1.8991428321988304, 0.1308280866492466,
0.7569740132498523, 10.888342020566464, 17.841900024979477]



```

/var/folders/xm/h9wx824x47311kz87v7c1g440000gn/T/ipykernel_54463/2294539193.py:4
5: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been
renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be
dropped in 3.11.
plt.boxplot(

```



```

/var/folders/xm/h9wx824x47311kz87v7c1g440000gn/T/ipykernel_54463/2294539193.py:6
0: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been
renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be
dropped in 3.11.
plt.boxplot(

```

